

Übungsblatt 2 – Santa Claus Problem

1. Einleitung und Problemstellung

Das „Santa Claus Problem“ ist ein klassisches Synchronisationsproblem, das komplexe Anforderungen an das Ressourcenmanagement und die Thread-Koordination stellt. Das Szenario beschreibt einen schlafenden Weihnachtsmann, der nur unter zwei Bedingungen geweckt wird:

- alle $r = 9$ Rentiere sind aus dem Urlaub zurückgekehrt
- eine Gruppe von $p = 3$ Elfen benötigt Hilfe bei der Spielzeugproduktion

Die zentrale Herausforderung besteht darin, Deadlocks zu vermeiden und die korrekte Priorisierung sicherzustellen. Die Auslieferung der Geschenke (wenn alle Rentiere bereit sind) hat Vorrang vor den Problemen der Elfen. Zudem muss sichergestellt werden, dass Elfen nicht einzeln, sondern nur in der definierten Gruppengröße bedient werden. Dieser Bericht dokumentiert zwei unterschiedliche Lösungsansätze für dieses Problem: eine lokale Implementierung mittels Semaphoren und eine verteilte Implementierung mittels Docker-Containern und ZeroMQ.

2. Aufgabe 1: Semaphore-basierte Implementierung

2.1 Konzept

Die erste Lösung basiert auf Shared-Memory-Kommunikation innerhalb eines einzelnen Prozesses. Zur Synchronisation wurden ausschließlich Semaphore verwendet, angelehnt an die Lösungsvorschläge aus „The Little Book on Semaphore“.

Die Architektur besteht aus drei Agenten-Typen, die als Threads realisiert wurden:

- **Santa-Thread:** Wartet passiv auf ein Signal (*santa_sem*)
- **Rentier-Threads ($r = 9$):** Inkrementieren einen Zähler und signalisieren Santa, nur wenn das letzte Rentier eintrifft
- **Elfen-Threads ($e = 10$):** Nutzen einen Mutex, um sich zu Gruppen zusammenfinden

2.2 Synchronisationslogik

Um die Gruppenbildung der Elfen zu steuern, wurde ein Zähler (*elfen_counter*) durch einen Mutex (*elf_tex*) geschützt. Ein zusätzlicher Mutex (*elf_mutex*) stellt sicher, dass keine weiteren Elfen versuchen, in die „Sprechstunde“ zu platzen, solange die aktuelle Dreiergruppe noch bedient wird.

Der Weihnachtsmann prüft nach dem Aufwachen zuerst den Status der Rentiere. Dies implementiert die geforderte Priorität: Sollten Rentiere und Elfen gleichzeitig bereitstehen, werden die Rentiere bevorzugt behandelt, da die Auslieferung der Geschenke den kritischen Pfad darstellt.

3. Aufgabe 2: Verteilte Architektur mit Docker & ZeroMQ

Der Schwerpunkt dieser Aufgabe lag auf der Modellierung des Problems als verteiltes System. Hierbei wurden die Akteure in isolierte Docker-Container ausgelagert, wodurch 1 + r + e unabhängige Instanzen entstanden. Da Docker-Container keinen gemeinsamen Arbeitsspeicher teilen, musste der Synchronisationsmechanismus von Semaphoren auf *Message Passing* umgestellt werden.

3.1 Architektur und Kommunikation (ZeroMQ)

Für die Kommunikation wurde die Bibliothek **ZeroMQ** gewählt, da sie eine hochperformante, nachrichtenbasierte Synchronisation ermöglicht.

Anstatt klassischer Semaphoren wurde das **ROUTER-REQ Pattern** implementiert:

- **Server (Santa):** Bindet einen *Router*-Socket an Port 5555. Dieser Socket-Typ ermöglicht asynchrone Kommunikation. Santa kann Nachrichten von vielen Clients empfangen, deren Identität speichern, aber die Antwort beliebig lange verzögern.
- **Clients (Elfen/Rentiere):** Nutzen *REQ*-Sockets. Ein `socket.send()` blockiert logisch bzw. erwartet zwingend ein `socket.recv()`. Dies simuliert exakt das Verhalten eines *Semaphor-acquire()*: Der Client sendet „Ich bin da“ und blockiert, bis Santa mit „Komm rein“ (Release) antwortet

3.2 Implementierungsdetails

Der Status des Systems (Anzahl wartender Elfen/Rentiere) wird nicht mehr in globalen Variablen, sondern im lokalen Speicher des Santa-Containers gehalten.

Der Weihnachtsmann fungiert als zentraler Server und nutzt das **ROUTER-Socket-Pattern**.

- **Addressierung:** Der ROUTER-Socket liest Multipart-Messages im Format `[Identity, Empty, Content]`. Die *Identity* wird von ZeroMQ automatisch generiert (oder entspricht der Docker-IP/ID) und ermöglicht es Santa, Clients individuell anzusprechen
- **State Management:** Anstatt globale Counter zu nutzen, verwaltet Santa zwei Listen: `waiting_elves[]` und `waiting_reindeer[]`. Eingehende Nachrichten führen dazu, dass die Identität des Absenders (`msg[0]`) in die entsprechende Liste hinzugefügt wird.

- **Verzögerte Antworten:** Das Kernkonzept dieser Lösung ist, dass auf ein *recv()* sofort ein *send()* folgt. Santa speichert die Anfrage, sendet aber **keine** Antwort. Dadurch bleibt der Client im *recv()*-Zustand blockiert („schläft“).
- **Flush-Logik:** Erst wenn *len(waiting_reindeer) == 9* ist, iteriert Santa über die Liste und sendet an alle gespeicherten Identitäten das Signal „GO_FLY“

Die Clients (Elfen/Rentiere) nutzen **REQ-Sockets** (Request). Dieser Socket-Typ erzwingt eine strikte Send-Receive-Abfolge.

Ablauf der Elfen-Gruppe:

1. Elf-Container sendet *ELF_HILFE* an Santa
2. Santa empfängt die Nachricht und speichert die ID des Absenders in einer Liste *waiting_elves*
3. Santa prüft: *if len(waiting_elves) < 3* : Keine Antwort senden. Der Elf-Client blockiert weiter.
4. Sobald der 3. Elf eintrifft: Santa entnimmt drei IDs aus der Liste und sendet an diese spezifischen IDs die Nachricht *GO_WORK*
5. Die Elfen-Container empfangen die Nachricht, „arbeiten“ (simuliert durch *sleep*) und kehren in den Zyklus zurück

Ablauf der Rentier-Gruppe:

1. Ein Rentier-Container erwacht aus dem „Urlaub“ (*sleep*) und sendet die Nachricht „*RENTIER_DA*“ via *REQ-Socket* an Santa. Anschließend ruft er *socket.recv()* auf und blockiert.
2. Santa empfängt die Nachricht über den *ROUTER-Socket*. Er speichert die eindeutige Identity des Rentier-Sockets in der Liste *waiting_reindeer*
3. Santa prüft den Füllstand der Liste:
 - Solange *len(waiting_reindeer) < 9*: Es erfolgt keine Antwort. Der Rentier-Client verbleibt im blockierten Zustand (Warteposition)
4. Sobald das 9. Rentier eintrifft, ist die Bedingung erfüllt.
 - Santa iteriert über die komplette Liste *waiting_reindeer*
 - Er sendet nacheinander an alle 9 gespeicherten Identitäten das Signal „GO_FLY“
 - Die Liste wird geleert und Santa wechselt in den Zustand „Auslieferung“ (simuliert durch *sleep(2)*), in dem er keine weiteren Anfragen bearbeitet.
5. Die Rentier-Container empfangen das „GO_FLY“-Signal, lösen ihre Blockade und beginnen ihren Zyklus von neuem („Urlaub“).

3.3 Orchestrierung mit Docker Compose

Um die Anforderung des automatisierten Starts zu erfüllen, wurde die **Docker Compose** verwendet. Die Datei *docker-compose.yml* definiert drei Services: *santa*, *rentier* und *elf*.

Ein entscheidender Vorteil von Docker Compose ist die Skalierbarkeit über das *scale* (bzw. *replicas*) Attribut. Dies ermöglichte das Starten von exakt 9 Rentier- und 10 Elfen-Containern mit einem einzigen Befehl:

```
docker compose up --build --scale rentier=9 --scale elf=10
```

Obwohl Kubernetes eine breitere Anwendbarkeit auf mehreren Hosts ermöglichen würde, erwies sich Docker Compose für die lokale Simulation als effizienter und ausreichend, um die Konzepte der Container-Orchestrierung und Netzwerk-Isolation zu demonstrieren.

4. Validierung und Ergebnisse

Die Korrektheit der verteilten Implementierung wurde anhand der Container-Logs verifiziert. Die Log -Ausgaben belegen, dass die Synchronisationsbedingungen strikt eingehalten werden.

Beobachtung 1: Gruppenbildung der Elfen

Aus den Logs ist ersichtlich, dass Santa die Anfragen akkumuliert und erst bei Erreichen der Gruppengröße reagiert:

```
santa-1      | [Santa] Elf braucht Hilfe (1 wartend)
santa-1      | [Santa] Rentier angekommen (1/9)
santa-1      | [Santa] Elf braucht Hilfe (2 wartend)
santa-1      | [Santa] Elf braucht Hilfe (3 wartend)
santa-1      | [Santa] --- Hilfe Gruppe von 3 Elfen ---
```

Erst nach der Zeile „Helfe Gruppe“ erhalten die Elfen-Clients ihre Antwort (*GO_WORK*), was das korrekte Blockieren der Clients beweist.

Beobachtung 2: Vollzähligkeit der Rentiere

Ebenso warten die Rentiere, bis alle 9 Instanzen ihre Ankunft signalisiert haben:

```
rentier-1 | [062beba4150e] Zurück aus dem Urlaub. Melde mich  
bei Santa...
```

```
santa-1   | [Santa] Rentier angekommen (1/9)
```

```
rentier-9 | [979c976c49c8] Zurück aus dem Urlaub. Melde mich  
bei Santa...
```

```
rentier-4 | [f4c7fcfabae] Zurück aus dem Urlaub. Melde mich  
bei Santa...
```

```
santa-1   | [Santa] Rentier angekommen (2/9)
```

```
santa-1   | [Santa] Rentier angekommen (3/9)
```

```
rentier-3 | [ddfc48ed4fa0] Zurück aus dem Urlaub. Melde mich  
bei Santa...
```

```
santa-1   | [Santa] Rentier angekommen (5/9)
```

```
rentier-2 | [ac30477fcdb9] Zurück aus dem Urlaub. Melde mich  
bei Santa...
```

```
rentier-5 | [9b6f57d7930d] Zurück aus dem Urlaub. Melde mich  
bei Santa...
```

```
rentier-8 | [00e2c07d514e] Zurück aus dem Urlaub. Melde mich  
bei Santa...
```

```
rentier-7 | [cd2f42ab6d3f] Zurück aus dem Urlaub. Melde mich  
bei Santa...
```

```
santa-1   | [Santa] Rentier angekommen (6/9)
```

```
santa-1   | [Santa] Rentier angekommen (7/9)
```

```
santa-1   | [Santa] Rentier angekommen  
(8/9)
```

```
rentier-6 | [2c0ff55c463c] Zurück aus dem Urlaub. Melde mich  
bei Santa...
```

```
santa-1   | [Santa] Rentier angekommen (9/9)
```

```
santa-1   | [Santa] --- Alle 9 Rentiere da! ---
```

Dies bestätigt, dass der „Abflug erst stattfindet, wenn die Bedingung $r = 9$ erfüllt ist. Da alle Container gleichzeitig kontinuierlich in hoher Frequenz Logs an das Hauptterminal senden, kann es vorkommen, dass manche Logs früher angezeigt werden, obwohl die Ausführung nach der korrekten Logik folgt.

5. Vergleich und Fazit

Die Bearbeitung der beiden Aufgaben zeigte deutlich die Unterschiede zwischen den Methoden *Shared Memory* und *Message Passing*.

Die Container-Lösung ist im Gegensatz zur Thread-Lösung horizontal skalierbar. Während die Python-Threads an eine einzelne Maschine gebunden sind, könnten die Docker-Container (insbesondere bei Verwendung von Kubernetes) auf verschiedene physische Server verteilt werden, ohne dass der Code geändert werden müsste. Das ZeroMQ-Protokoll abstrahiert die Netzwerkschicht dabei vollständig.

Zusammenfassend konnte das Santa Claus Problem erfolgreich in eine Microservices-ähnliche Architektur überführt werden, die robust und deadlock-frei arbeitet.